

ETSI GS QKD 014 on the wire – a hands-on protocol analysis

Author: Rodolphe Masson | **Date:** August 2026 | **Keywords:** QKD, ETSI-QKD-014, Wireshark, mTLS, TLS13, quantum-safe, post-quantum, QuKayDee Platform

Executive Summary

The ETSI GS QKD 014 key delivery API [2] was exercised end to end and captured at packet level, against the QuKayDee hosted QKD network simulator [1]. Ten test cases cover the nominal key delivery flow, the key store post-condition, parameter validation, client identity binding, the cryptographic posture of the channel, failure condition discrimination, connection management, key store exhaustion, the POST request form with its extension mechanism, and multicast delivery.

The interface behaves as specified. A key issued by kme-1 to the master SAE was retrieved byte-identical from kme-2 by the slave SAE, with only the key identifier passing between them. A delivered key was removed from the store and could not be fetched twice. Parameters were rejected against the limits the KME advertises. Three invalid client identities were denied and no unauthorised key material was released. An extension marked mandatory but unsupported was refused, as the standard requires. The one explicit obligation tested, and it holds.

Four things stand out from the traces:

1. **Access control is a certificate and nothing else.** No token, password or session exists anywhere in the API; the SAE's identity is established in the mutual TLS handshake and never restated. In this implementation the TLS layer did not make the decision – all four test identities completed the handshake, including one signed by an unknown certificate authority, and every rejection came from the application behind it. The standard permits either arrangement, so the trust boundary may sit in different places in different products.
2. **The channel carrying the key material is classical.** The client offered the post-quantum hybrid group X25519MLKEM768; the server selected X25519, and none of the sixteen signature algorithms offered for client authentication were post-quantum. ETSI GS QKD 014 imposes no requirement here, so this is conformant.
3. **The standard mandates behaviour it provides no way to report.** A delivered key must leave the key store, and an expired key must not be served – yet the status codes carrying both conditions lie outside the set the specification defines. Key store depletion is reported with a code whose semantics instruct a client not to retry, when retrying after a delay is precisely correct. Each implementation must invent its own signalling for behaviour the standard requires.
4. **The key store is less visible than it appears.** It is accounted in bits while the interface exposes only derived key counts; a saturated store discarding generated material gives no indication that it is doing so; and a key designated for several slave SAEs is not released on first delivery – a case the standard does not address at all.

On the standard itself. ETSI GS QKD 014 defines what a key store must *do* at the application boundary and says nothing about what it must *be*: its state, its synchronisation with a peer, and its

behaviour after a link fault are unspecified. The operational layer lives in separate deliverables, and ETSI GS QKD 020 [5], covering key management system interoperability, was published in June 2026.

One practical observation. No reference implementation, conformance suite or ready-to-run container accompanies the standard. This study was possible because a third party built and hosts a simulator. Its absence has a measurable cost: three of the ten test cases returned results contradicting what had been inferred from careful reading of the specification. A published specification is a necessary condition for interoperability; it has never been a sufficient one.

Scripts, packet captures and figures: <https://github.com/Rodolphe21/qkd-etsi014-poc>

Results Summary

The table records the outcome of each test case. **Pass** indicates the observed behaviour matched the expectation stated in the test case; **characterisation** indicates a test case with no pass criterion, whose output is a record of one implementation's behaviour. Three test cases returned results contradicting the expectation and are marked accordingly.

TC Objective	Outcome	Finding
01 Cross-KME key delivery integrity	Pass	A key issued by kme-1 to sae-1 was retrieved byte-identical from kme-2 by sae-2, with only the identifier passing between them.
02 Key consumption and replay rejection	Pass	A delivered key is removed from the store; the identifier alone confers no further access. Qualified by TC-10 for the multicast case.
03 Request parameter validation	Pass	The capability set advertised in <i>Get status</i> is binding. Errors name the parameter, echo the value and state the permitted range.
04 SAE identity binding and authorisation layering	Contrary to expectation	All four identities completed the mutual TLS handshake, including one signed by an unregistered CA. Every rejection occurred at the application layer; no transport-layer rejection was observed.
05 TLS cryptographic posture	Characterisation	The client offered X25519MLKEM768; the server selected x25519. Sixteen signature algorithms were offered for client authentication, none post-quantum. Conformant – the standard imposes no requirement.
06 Failure condition discrimination	Contrary to expectation	Consumed and never-issued are indistinguishable (404); expiry is reported explicitly (403); a malformed identifier is rejected by parameter validation (400). Neither 403 nor 404 appears in the status codes the specification defines.

TC Objective	Outcome	Finding
07 Connection reuse and handshake cost	Contrary to expectation	Reuse is supported: seven handshakes served eleven requests. Median handshake 241 ms against median request 407 ms.
08 Key store exhaustion and replenishment	Pass, with a semantic finding	Depletion returns 400 "Not enough key material available" – a defined code, but one instructing a client not to retry a request that should be retried. No partial delivery. Replenishment linear at 997 bits/s against 1000 configured. The store is accounted in bits; stored_key_count is derived.
09 POST form and extension handling	Pass	An unknown extension_mandatory is refused and an unknown extension_optional ignored, as the standard requires. Batch retrieval of several identifiers in one request is supported.
10 Multicast key delivery	Pass	additional_slave_SAE_IDs is honoured: byte-identical key material was delivered to two slave SAEs at two further KMEs. Retrieval by the first slave did not release the key.

Three test cases returned results contradicting the expectation recorded in them. The corrections are retained in the text rather than removed, and the expectations are stated as they were held.

Context

ETSI GS QKD 014 [2] defines a REST interface between a Key Management Entity and the applications consuming its keys: three methods and one key container format, with a request body in the POST form carrying multicast and vendor-extension parameters, deliberately narrower than the session-oriented ETSI GS QKD 004 [6]. The operational layer lives in separate deliverables: ETSI GS QKD 015 (SDN) [3], ETSI GS QKD 018 [4] (orchestration), ETSI GS QKD 020 [5] (KMS interoperability, published June 2026). The second edition of ETSI GS QKD 014 is in the ETSI work programme.

ETSI GS QKD 014 [2] is nevertheless the interface an encryptor or VPN gateway actually talks to, and the point where quantum-derived key material crosses into classical infrastructure, which makes it the natural place to look from the consuming side.

The analysis is conducted at the packet level against a hosted simulator called QuKayDee [1], with the TLS sessions decrypted so that the API exchanges and the underlying handshake are both visible. Findings concerning message structure, error behaviour and the identity model follow from the specification; specific parameter values and the observed TLS posture characterize this implementation only. A third class of finding concerns behaviour the specification neither defines nor constrains. Where this report records such a case, the observation is that the standard leaves the choice open, not that any particular choice is correct.

GitHub Repository Content

```
qkd-etsi014-poc/
├── README.md
├── docs/
│   ├── qkd-etsi014-report.docx      this report
│   └── img/                         figures
├── src/
│   ├── qukaydee_etsi014.py          TC-01, TC-02, TC-03, TC-05
│   ├── tc04_identity.py              TC-04
│   ├── tc06_failure_conditions.py    TC-06
│   ├── tc07_connection_reuse.py      TC-07
│   ├── tc08_exhaustion.py            TC-08
│   ├── tc09_post_extensions.py       TC-09
│   └── tc10_multicast.py             TC-10
├── captures/
│   ├── tc01-03-key-delivery.pcapng
│   ├── tc04-identity.pcapng
│   ├── tc06-failure-conditions.pcapng
│   ├── tc07-connection-reuse.pcapng
│   ├── tc08-exhaustion.pcapng
│   ├── tc09-post-extensions.pcapng
│   ├── tc10-multicast.pcapng
│   └── qukaydee-keys.log             TLS session keys for the above
└── .gitignore
```

Artefact	Covers
qukaydee_etsi014.py	Cross-KME key delivery, replay rejection, parameter validation
tc04_identity.py	Four client certificates against one endpoint
tc06_failure_conditions.py	Consumed, expired, unknown and malformed identifiers
tc07_connection_reuse.py	Connection reuse and handshake cost
tc08_exhaustion.py	Key store depletion and replenishment rate
tc09_post_extensions.py	POST form, batch retrieval, extension handling
Tc10_multicast.py	Multicast delivery to two slave SAEs
*.pcapng	Packet captures backing every figure
qukaydee-keys.log	Session keys permitting independent decryption of the captures

Not included. SAE private keys and CA key material are excluded from version control; the scripts require certificates generated locally as described in Section 1. Reproduction also requires a QuKayDee account and the topology described there, including the extended topology for TC-10, and additional identities for TC-04 (sae-9, and a certificate signed by an unregistered CA). The account identifier is a command-line argument to every script.

On the key log. The TLS session key log is committed deliberately, so that the captures can be decrypted and the claims in this report verified independently rather than taken from the screenshots. The sessions are closed, the account is a test account, and the platform states its keys are not cryptographically secure. This would not be appropriate for a capture of production traffic.

1. Environment & Tools

Platform under test

QuKayDee [1]: hosted QKD network simulator exposing an ETSI GS QKD 014 V1.1.1 interface. Account 4627.

Topology configured through the web portal:

Element	Configuration
KMEs	kme-1 (KME Alice), kme-2 (KME Bob)
SAEs	sae-1 → KME Alice, sae-2 → KME Bob. DN left at default CN=sae-<n>
Key stream	sae-1 ↔ sae-2, 1000 bits/sec, key expiry 600 s
Extended topology (TC-10)	kme-3 (KME Charlie), sae-3 → KME Charlie, second key stream spanning sae-1, sae-2 and sae-3, stream ID 387. See Figure 1b.

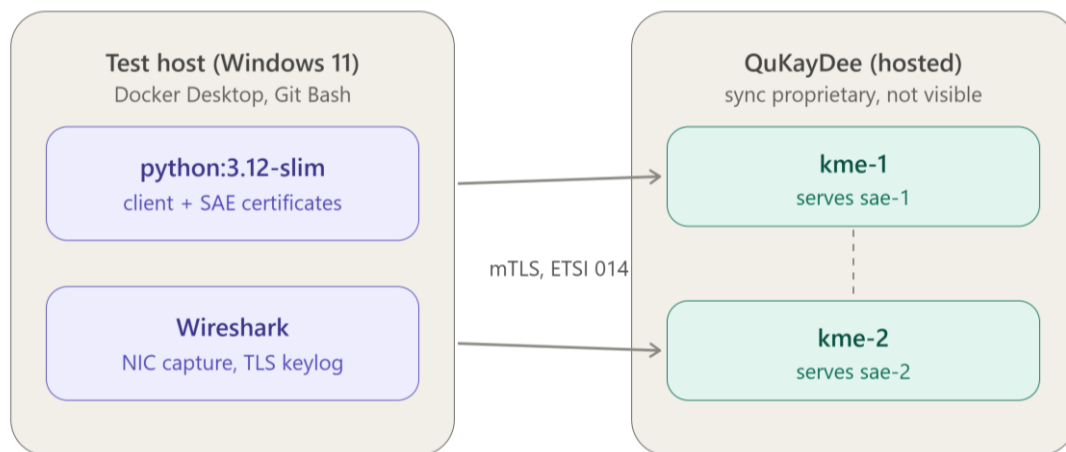


Figure 1 – PoC topology. The client presents sae-1 to kme-1 and sae-2 to kme-2, each over a separate mutual TLS session. Only the key ID passes between the two SAE identities, and that transfer happens inside the client script (not defined in the specification). The dashed link is the KME-to-KME synchronisation: it carries the key material that makes cross-KME retrieval possible, but it is proprietary and not exposed by the platform, so it falls outside the capture.

Test cases TC-01 to TC-09 use this topology. TC-10 requires a third KME and SAE; the extended topology is shown in Figure 1b.

Endpoints: <https://kme-{1,2}.acct-4627.etsi-qkd-api.qukaydee.com/api/v1/keys/...>

Key store state observed at test time:

key_size	512
----------	-----

stored_key_count	1953 (equal to max_key_count; store is saturated)
max_key_count	1953
max_key_per_request	100
min_key_size	1
max_key_size	100000
max_SAE_ID_count	9
max_key_count × key_size	999 936 bits against a store capacity of 1 000 000 bits. The key store is accounted in bits; stored_key_count and max_key_count are derived figures. See TC-08.

Client

Custom Python 3.12 client (qukaydee_etsi014.py), standard library only, urllib.request over an ssl.SSLContext carrying the SAE certificate and the platform CA. The TLS session-key log path is set explicitly via ctx.keylog_filename rather than the SSLKEYLOGFILE environment variable.

Executed inside the stock python:3.12-slim container:

```
> docker run --rm -v "$(pwd):/work" -w /work python:3.12-slim python qukaydee_etsi014.py 4627
```

Rationale. The Windows-native and Git Bash builds of curl on the test host are linked against Schannel, which does not implement TLS key logging – a prerequisite for decrypting the capture. CPython bundles OpenSSL, so running the client in a Linux container removed the host TLS stack from the equation entirely. The same image also provided a Linux bash and openssl for certificate generation, avoiding MSYS path rewriting and CRLF issues.

Certificates

Generated with brunorijsman/qukaydee-generate-client-certificates [7], executed in the container after stripping CRLF line endings from the scripts.

- client-root-ca.crt / .key — client root CA, public certificate uploaded to the platform
- sae-1.crt / .key, sae-2.crt / .key — SAE client certificates, retained locally
- account-4627-server-ca-qukaydee-com.crt — platform server CA, downloaded from the portal

The SAE private keys are excluded from version control.

Capture and Analysis

Wireshark on the host's physical network adapter (traffic is outbound to the internet: WLAN).

Decryption: *Preferences* → *Protocols* → *TLS* → *(Pre)-Master-Secret log filename* pointed at qukaydee-keys.log.

Display filters used:

Purpose	Filter
Mutual authentication exchange	tls.handshake.type in {11, 13, 15}
Negotiated key agreement group	tls.handshake.type == 2 && tls.handshake.extensions_key_share_group
Decrypted API traffic	http

Host

Windows 11, Docker Desktop, Git Bash.

MSYS_NO_PATHCONV=1 required on docker run invocations to prevent MSYS rewriting container paths.

2. Test Case TC-01 – Cross-KME key delivery integrity

Objective. Verify the core assertion of ETSI GS QKD 014: key material issued by one KME to the master SAE is retrievable, bit-identical, from a *different* KME by the slave SAE, with only the key ID passing between them.

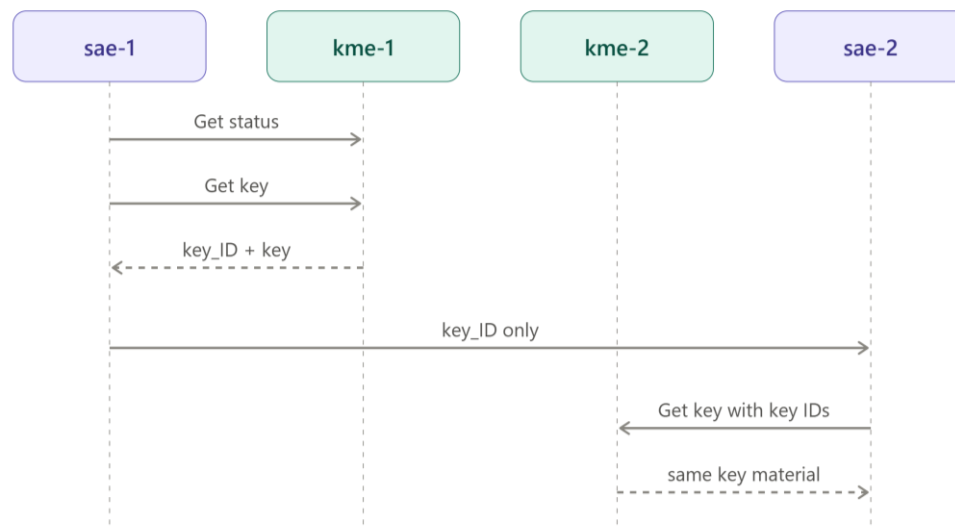


Figure 2 – Expected message flow, ETSI GS QKD 014. Solid arrows are requests, dashed are responses. The transfer of the key identifier from master to slave SAE is not defined by the standard and is left to the application; here it is a variable assignment within the test client. (No message passes between the two KMEs in this flow. The key material they share is established beforehand by mechanisms outside the scope of the standard.)

Preconditions. Topology live, key stream producing, stored_key_count > 0 on kme-1. Both SAE certificate pairs present and the client CA registered on the platform.

Procedure.

#	Action	Endpoint	Identity
1	Get status	kme-1 /api/v1/keys/sae-2/status	sae-1
2	Get key	kme-1 /api/v1/keys/sae-2/enc_keys number=2 & size=1024	sae-1
3	Transfer key ID only, out of band	—	—
4	Get key with key IDs	kme-2 /api/v1/keys/sae-1/dec_keys key_ID = <id>	sae-2
5	Byte-compare the two key values	—	—

Expected result. Both calls return HTTP 200. The base64 key strings are identical. No key material traverses the path between the two SAE identities.

Observed result. Pass.

Step 1 returned the key store capability set:

json

```
{"source_KME_ID": "kme-1", "target_KME_ID": "kme-2",  
  "master_SAE_ID": "sae-1", "slave_SAE_ID": "sae-2",  
  "key_size": 512, "stored_key_count": 1839, "max_key_count": 1953,  
  "max_key_per_request": 100, "max_key_size": 100000, "min_key_size": 1,  
  "max_SAE_ID_count": 9}
```

Step 2 returned two key objects. The first carried key_ID 4449fe18-f661-42cb-8480-a19f79a53116 with 1024 bits of base64-encoded material.

```
GET /api/v1/keys/sae-2/enc_keys?number=2&size=1024 HTTP/1.1
```

```
Accept-Encoding: identity
```

```
Host: kme-1.acct-4627.etsi-qkd-api.qukaydee.com
```

```
User-Agent: Python-urllib/3.12
```

```
Accept: application/json
```

```
Connection: close
```

```
HTTP/1.1 200 OK
```

```
Server: nginx/1.18.0 (Ubuntu)
```

```
Date: Wed, 05 Aug 2026 14:50:20 GMT
```

```
Content-Type: application/json
```

```
Content-Length: 472
```

```
Connection: close
```

```
Vary: Accept, Cookie
```

```
Allow: GET, POST, HEAD, OPTIONS
```

```
[...]
```

```
{"keys":[{"key_ID":"4449fe18-f661-42cb-8480-a19f79a53116","key":"Fy7k1XC ...  
rddSj0="},{key_ID":"38673 ... f06","key":"EbSlb2n ... 4Es="}]}
```

Figure 3 – Get key. GET request and response captured between sae-1 and kme-1, decrypted using the TLS session key log. The response body is the ETSI 014 key container: two objects, each carrying a key_ID and 1024 bits of base64-encoded key material.

Step 4, issued against the second KME under the second SAE identity, returned a single key object bearing the same key_ID and byte-identical key value. Comparison at step 5 returned true.

```

GET /api/v1/keys/sae-1/dec_keys?key_ID=4449fe18-f661-42cb-8480-a19f79a53116
HTTP/1.1
Accept-Encoding: identity
Host: kme-2.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Connection: close

HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 05 Aug 2026 14:50:21 GMT
Content-Type: application/json
Content-Length: 241
Connection: close
Vary: Accept, Cookie
Allow: GET, POST, HEAD, OPTIONS
[...]
{"keys":[{"key_ID":"4449fe18-f661-42cb-8480-a19f79a53116","key":"Fy7k1XC ...
lrddSj0="}]}

```

Figure 4 – Get key with key IDs. Response from kme-2 under the sae-2 identity. Same key_ID and same key material as Figure 3, retrieved from the second KME. Both calls completed within one second of each other (14:50:20 and 14:50:21), against different hosts, under different client certificates.

Notes.

The request at step 2 specified size=1024 while status advertised key_size: 512. The request succeeded. key_size is therefore a default rather than a constraint; the binding limits are min_key_size and max_key_size.

Step 3 is the point at which the specification stops. ETSI GS QKD 014 defines no mechanism for conveying the key ID from master to slave: that channel is left entirely to the application. In this test it is a variable assignment inside the client script; in a deployment it is whatever the two SAEs already share. The security argument rests on the key ID being non-secret: it is an identifier, and possession of it confers nothing without an authorised mTLS session to the KME holding the key.

The two KMEs must therefore already hold common key material before step 2 is issued. That synchronisation is not visible at this interface and is not exercised by this test case (see Figure 1).

3. Test Case TC-02 – Key consumption and replay rejection

Objective. Verify the key store post-condition of ETSI GS QKD 014 [2]: a key delivered to the slave SAE is removed from the store and cannot be retrieved a second time. Determine whether a consumed key is distinguishable from one that never existed. Note that the test client is programmed to issue a second *Get key with key IDs* request with the same key ID, allowing this condition to be tested deliberately. The 404 observed below is therefore constructed, not an operational event.

Preconditions. TC-01 executed successfully. The key_ID retrieved at TC-01 step 4 available, and known to have been delivered.

Procedure.

#	Action	Endpoint	Identity	Parameter
1	Replay the successful retrieval	kme-2 /api/v1/keys/sae-1/dec_keys	sae-2	key_ID from TC-01
2	Request a never-issued identifier (not executed, see below)	kme-2 /api/v1/keys/sae-1/dec_keys	sae-2	key_ID=00000000-0000-0000-0000-000000000000
3	Compare the two responses	-	-	-

Expected result. Step 1 returns an error; the key is no longer available. Step 2 returns an error. If the two responses are indistinguishable, the interface exposes no information about which identifiers have ever existed.

Observed result. Pass on step 1.

```
GET /api/v1/keys/sae-1/dec_keys?key_ID=4449fe18-f661-42cb-8480-a19f79a53116 HTTP/1.1
Accept-Encoding: identity
Host: kme-2.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Connection: close

HTTP/1.1 404 Not Found
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 05 Aug 2026 14:50:21 GMT
Content-Type: application/json
Content-Length: 89
Connection: close
Vary: Accept, Cookie
Allow: GET, POST, HEAD, OPTIONS
[...]
{"message": "Key not found", "details": [{"key_ID": "4449fe18-f661-42cb-8480-a19f79a53116"}]}
```

Figure 5 – Replay of a delivered key identifier. The same GET as Figure 4, issued a second time to kme-2 under the same identity, returns HTTP 404 "Key not found". The key was removed from the store on delivery.

The replay returned HTTP 404:

```
Json
{"message": "Key not found",
 "details": [{"key_ID": "4449fe18-f661-42cb-8480-a19f79a53116"}]}
```

The key delivered in TC-01 was therefore removed from the key store on delivery, and the identifier alone confers no further access.

Step 2 was not executed in this run. It was subsequently carried out in TC-06, together with two further failure conditions; the comparison is reported there.

Notes.

Returning the same 404 for a consumed key as for an unknown identifier is sound security design: it denies an attacker an oracle for which identifiers have ever been issued. TC-06 confirms that the two are in fact indistinguishable.

The consequence for operations is more limited than the single response suggests and is set out in TC-06. A consumed key and an identifier that never existed are indistinguishable, as intended; an expired key is reported explicitly with a different status code and message. What remains indistinguishable from a consumed key, in a deployment where the two KMEs are separate systems, is a key whose assignment has not propagated between them, a condition that cannot arise on the platform used here and is therefore untested. The only key store observable in normal operation remains the `stored_key_count` scalar in *Get status*.

The post-condition demonstrated here applies to a key designated for a single slave SAE. Where several slaves are designated, the key is not removed on first delivery; see TC-10.

4. Test Case TC-03 – Request Parameter Validation

Objective. Verify that a KME validates the parameters of a *Get key* request against the limits it advertises in *Get status*, and that rejections identify the offending parameter. The property under test is that the advertised capability set is binding rather than informational. The test client derives these values from the *Get status* response and deliberately sends parameters one increment beyond each advertised limit. The rejections observed below are therefore constructed.

Preconditions. *Get status* executed against kme-1 and its response recorded, to derive the test values.

Procedure. Issue *Get key* against kme-1 as sae-1, with parameters derived from the values returned by *Get status*:

#	Parameter	Advertised limit	Value sent
1	size	max_key_size 100000	100008
2	number	max_key_per_request 100	101

Expected result. Both requests rejected with a 4xx status and an error identifying the offending parameter.

Observed result. Pass.

```
GET /api/v1/keys/sae-2/enc_keys?number=1&size=100008 HTTP/1.1
Accept-Encoding: identity
Host: kme-1.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Connection: close

HTTP/1.1 400 Bad Request
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 05 Aug 2026 14:50:22 GMT
Content-Type: application/json
Content-Length: 151
Connection: close
Vary: Accept, Cookie
Allow: GET, POST, HEAD, OPTIONS
[...]
{"message": "Invalid parameter
value","details":[{"parameter_name":"size","parameter_value":"100008","allowed_values":"Integer, between
1 and 100000"}]}
```

Figure 6 - Parameter validation. A *Get key* request exceeding the `max_key_size` advertised in *Get status* is rejected with HTTP 400. The error object names the parameter, echoes the rejected value, and states the permitted range.

Both returned HTTP 400:

```
Json

{"message": "Invalid parameter value",
 "details": [{"parameter_name": "size",
               "parameter_value": "100008",
```

```
"allowed_values": "Integer, between 1 and 100000"}}}
```

The response for number followed the same structure, citing the range 1 to 100. The advertised limits are therefore binding, and the error object identifies the parameter and its permitted range.

Notes.

The advertised limits are enforced, and the error object is well formed: it names the parameter, echoes the rejected value, and states the permitted range.

The contrast with retrieval failures is worth recording. Parameter errors identify precisely what was wrong; a retrieval failure returns the same response for a consumed key as for an identifier that never existed, deliberately so. TC-06 examines this in detail and finds the discrimination less uniform than it first appears: expiry is reported explicitly, a malformed identifier is rejected by parameter validation, and only the consumed and never-issued cases are indistinguishable. Input validation is designed for the client developer, and failure reporting for the security boundary.

Not exercised. Values below `min_key_size`, sizes that are not multiples of 8, and malformed or absent parameters. A malformed `key_ID` is exercised in TC-06. The POST form of the request, carrying `additional_slave_SAE_IDs` and the `extension_mandatory` / `extension_optional` fields, is exercised in TC-09 and TC-10.

5. Test Case TC-04 – SAE identity binding and authorisation layering

Objective. Verify that authorisation in ETSI GS QKD 014 [2] rests on the mutual TLS client certificate and on nothing else and determine at which layer each class of invalid identity is rejected: TLS transport or ETSI 014 application. The property under test is the location of the trust boundary, not merely the outcome.

Note that the test client issues the identical request four times, varying only the client certificate presented. All rejections observed below are therefore constructed.

Preconditions.

- Topology live; sae-1 and sae-2 registered; client root CA uploaded to the platform.
- Two additional identities generated and deliberately **not** registered:
 - sae-9: signed by the registered client CA, no corresponding SAE on the platform.
 - rogue-sae-1: signed by a second CA that was never uploaded. Subject CN=sae-1 and issuer DN CN=client-root-ca are identical to the valid credential; only the signing key pair differs (verified by CA fingerprint comparison).

Procedure. The four cases are implemented in tc04_identity.py, provided in the GitHub repository. The script issues an identical *Get key* request four times, same KME, same endpoint, same parameters, varying only the client certificate presented during the mutual TLS handshake:

```
GET kme-1 /api/v1/keys/sae-2/enc_keys?number=1&size=256
```

Invocation:

```
> docker run --rm -v "$(pwd):/work" -w /work python:3.12-slim python tc04_identity.py 4627
```

Holding every other variable constant is what allows the outcome to be attributed to identity alone. The script reports, for each case, whether rejection occurred at the TLS transport layer or at the ETSI 014 application layer and prints a summary table.

Cases.

Case	Certificate	Signed by	Registered on platform	Condition under test
A	sae-1	registered client CA	yes	Control, authorised master SAE
B	sae-2	registered client CA	yes	Valid identity, not master for this stream
C	sae-9	registered client CA	no	Trusted chain, unregistered identity
D	rogue-sae-1	unregistered CA	no	Untrusted chain, subject and issuer DN identical to case A

Expected result. Case A succeeds. Cases B and C complete the TLS handshake and are rejected by the application. Case D is rejected during the handshake, with a TLS alert and no HTTP exchange.

Observed result. Partial: the security outcome was correct, the layering was not as expected.

Time	Source	Destination	Protocol	Length	tcp.stream	Info
1497 122.742994300	3.227.206.55	192.168.178.20	TLSv1.3	1466 25	A	Server Hello, Change Cipher Spec, Encrypted Ext
1498 122.742995400	3.227.206.55	192.168.178.20	TLSv1.3	241 25	A	Certificate Verify, Finished
1500 122.752010700	192.168.178.20	3.227.206.55	TLSv1.3	1262 25	A	Change Cipher Spec, Certificate, Certificate Ve
1517 123.378441600	3.227.206.55	192.168.178.20	TLSv1.3	1466 26	B	Server Hello, Change Cipher Spec, Encrypted Ext
1518 123.378443300	3.227.206.55	192.168.178.20	TLSv1.3	241 26	B	Certificate Verify, Finished
1520 123.386589200	192.168.178.20	3.227.206.55	TLSv1.3	1262 26	B	Change Cipher Spec, Certificate, Certificate Ve
1538 123.753116900	3.227.206.55	192.168.178.20	TLSv1.3	1466 27	C	Server Hello, Change Cipher Spec, Encrypted Ext
1539 123.753118100	3.227.206.55	192.168.178.20	TLSv1.3	241 27	C	Certificate Verify, Finished
1541 123.762697200	192.168.178.20	3.227.206.55	TLSv1.3	1262 27	C	Change Cipher Spec, Certificate, Certificate Ve
1558 124.133806900	3.227.206.55	192.168.178.20	TLSv1.3	1466 28	D	Server Hello, Change Cipher Spec, Encrypted Ext
1559 124.133809500	3.227.206.55	192.168.178.20	TLSv1.3	241 28	D	Certificate Verify, Finished
1561 124.141797100	192.168.178.20	3.227.206.55	TLSv1.3	1262 28	D	Change Cipher Spec, Certificate, Certificate Ve

Figure 7 – Certificate exchange in all four TC-04 sessions. Each contains a CertificateRequest from the KME and a Certificate and “Certificate Verify” from the client, including case D, whose certificate was signed by a CA unknown to the platform. Every session completed the handshake.

Time	Source	Destination	Protocol	Length	tcp.stream	Info
1501 122.753397400	192.168.178.20	3.227.206.55	HTTP	291 25	A	GET /api/v1/keys/sae-2/enc_keys?number=1&size=25
1504 123.148202500	3.227.206.55	192.168.178.20	HTTP/JSON	530 25	A	HTTP/1.1 200 OK , JSON (application/json)
1521 123.387635100	192.168.178.20	3.227.206.55	HTTP	291 26	B	GET /api/v1/keys/sae-2/enc_keys?number=1&size=25
1524 123.517726600	3.227.206.55	192.168.178.20	HTTP/JSON	517 26	B	HTTP/1.1 404 Not Found , JSON (application/json)
1542 123.764132700	192.168.178.20	3.227.206.55	HTTP	291 27	C	GET /api/v1/keys/sae-2/enc_keys?number=1&size=25
1546 123.897266800	3.227.206.55	192.168.178.20	HTTP/JSON	517 27	C	HTTP/1.1 404 Not Found , JSON (application/json)
1562 124.142992200	192.168.178.20	3.227.206.55	HTTP	291 28	D	GET /api/v1/keys/sae-2/enc_keys?number=1&size=25
1565 124.251597500	3.227.206.55	192.168.178.20	HTTP	461 28	D	HTTP/1.1 400 Bad Request (text/html)

Figure 8 – HTTP responses for the four TC-04 sessions, correlated by TCP stream to the handshakes in Figure 7. Case A succeeds; cases B and C are refused with 404; case D, presenting an unregistered certificate chain, is refused with 400. All four responses were served over a completed mutual TLS session.

Case	Rejection layer	Status
A	-	200
B	application	404
C	application	404
D	application	400

1558 124.133806900	3.227.206.55	192.168.178.20	TLSv1.3	1466 28	Server Hello, Change Cipher Spec, tm
1561 124.141797100	192.168.178.20	3.227.206.55	TLSv1.3	1262 28	Change Cipher Spec, Certificate, Cer

```

TLSv1.3 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
TLSv1.3 Record Layer: Handshake Protocol: Certificate
  Opaque Type: Application Data (23)
  Version: TLS 1.2 (0x0303)
  Length: 837
  [Content Type: Handshake (22)]
  [Sequence Number: 0]
  Handshake Protocol: Certificate
    Handshake Type: Certificate (11)
    Length: 816
    Certificate Request Context Length: 0
    Certificates Length: 812
    Certificates (812 bytes)
      Certificate Length: 807
      Certificate [...]: 308203233082020ba0030201020214165fd294d124698cd3aeba413128c112674df49a300d06092a864886f70d01010b05003019311730150603
        signedCertificate
          version: v3 (2)
          serialNumber: 0x165fd294d124698cd3aeba413128c112674df49a
          signature (sha256WithRSAEncryption)
          issuer: rdnSequence (0)
            rdnSequence: 1 item (id-at-commonName=client-root-ca)
          validity
          subject: rdnSequence (0)
            rdnSequence: 1 item (id-at-commonName=sae-1)
              RDNSequence item: 1 item (id-at-commonName=sae-1)
            subjectPublicKeyInfo
            extensions: 5 items
            algorithmIdentifier (sha256WithRSAEncryption)

```

Figure 9 – Client certificate presented in case D. Subject (CN=sae-1) and issuer (CN=client-root-ca) are identical to the valid credential of case A; the certificate was nonetheless signed by a CA never registered with the platform, confirmed separately by fingerprint comparison. It was accepted at the transport layer and the request rejected by the application.

No transport-layer rejection occurred. All four sessions completed the mutual TLS handshake, including case D. Filtering the capture for TLS alert records (`tls.record.content_type == 21`) returned only `close_notify` at warning level: normal session teardown and not a rejection.

The rogue credential was therefore presented, accepted at the transport layer, and rejected by the application, which returned HTTP 400: a different code from the 404 returned for cases B and C.

Notes.

The rogue certificate matches the valid credential on both subject and issuer distinguished name, differing only in the key pair that signed it. Its rejection demonstrates that the trust decision is a signature verification against the registered CA public key, not a comparison of any name in the certificate.

The observed layering indicates that TLS is terminated at a reverse proxy configured for *optional* client verification: a certificate is requested, the handshake completes irrespective of chain validation, and the verification outcome is passed to the application, which makes the authorisation decision. This is consistent with the Server: `nginx/1.18.0` header and the Django-style response headers observed throughout. It is a common production pattern, not a simulator shortcut. It allows a meaningful HTTP error in place of an opaque handshake failure, at the cost of moving the trust decision out of the TLS stack and into application code.

ETSI GS QKD 014 [2] delegates authentication entirely to TLS and PKI and specifies nothing about how it is to be enforced. Edge termination with optional verification, edge termination with mandatory verification, and verification within the key management logic itself are all conformant. Two implementations may therefore place the trust boundary in materially different places while both satisfying the standard. This test case is consequently reusable as a characterisation instrument: run unchanged against a vendor KME, it may return a different layering.

The distinction between the 400 of case D and the 404 of cases B and C suggests the application separates a certificate that failed validation from an identity that is not authorised. Cases B and C returning the same code preserves the no-oracle property observed in TC-02: the interface does not confirm which SAEs exist.

Security outcome. All three invalid identities were denied. No unauthorised key material was released. This test case reports a layering property, not a vulnerability.

6. Test Case TC-05 – TLS cryptographic posture of the key delivery channel.

Objective. ETSI GS QKD 014 [2] delegates channel protection and authentication entirely to TLS and PKI and specifies nothing about what TLS must negotiate. Determine what is actually negotiated on the key delivery channel: the protocol version, the key agreement group, and the signature algorithm used for client authentication, as *selected*, not as offered.

This test case is characterisation rather than pass/fail. Its output is a record of one endpoint's posture and a method that can be reapplied unchanged to any other ETSI 014 implementation.

Preconditions. A capture from any preceding test case, with the TLS session key log loaded and decryption confirmed. No new traffic is required.

Procedure. Five values read from the decrypted handshake:

#	Value	Location	Filter
1	Protocol version	ServerHello → supported_versions extension	tls.handshake.type == 2
2	Groups offered	ClientHello → supported_groups, key_share entries	tls.handshake.type == 1
3	Group selected	ServerHello → key_share → Group	tls.handshake.type == 2 && tls.handshake.extensions_key_share_group
4	Signature algorithms offered	CertificateRequest → signature_algorithms	tls.handshake.type == 13
5	Algorithm selected	Client CertificateVerify → Signature Algorithm	tls.handshake.type == 15

Steps 1 and 3 require care. TLS 1.3 writes the legacy value 0x0303 (TLS 1.2) into both the record layer and the ServerHello version field for compatibility with middleboxes that predate the version. The negotiated version is carried only in the supported_versions extension. Similarly, supported_groups in the ClientHello is the client's offer and says nothing about the connection; only the ServerHello key_share records what was used.

Observed result.

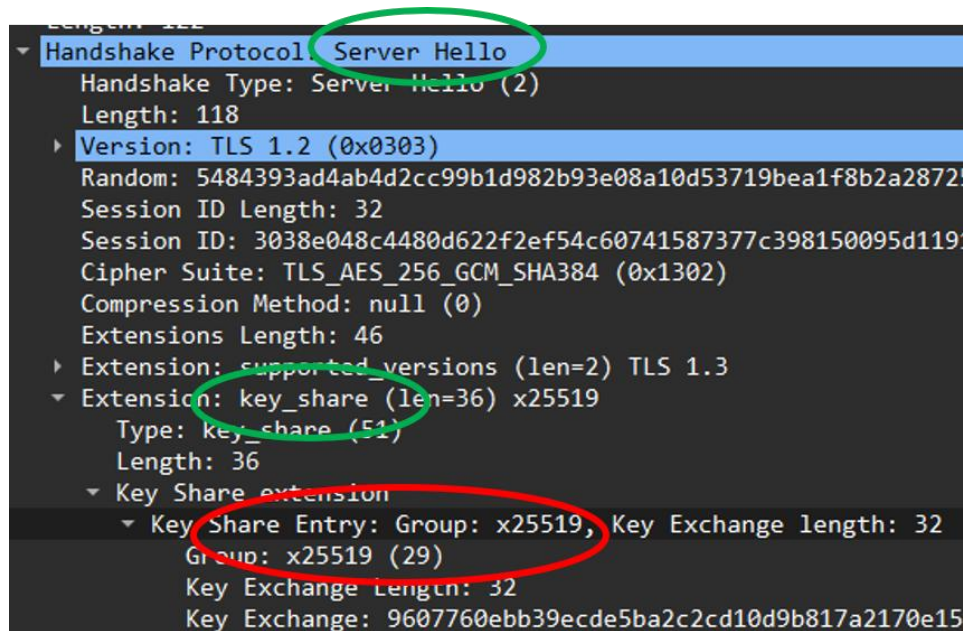


Figure 10 – Negotiated key agreement group. The ServerHello selects x25519. The client had offered X25519MLKEM768 alongside it and sent both key shares; the classical fallback was chosen.

#	Value	Observed
1	Protocol version	TLS 1.3
2	Groups offered	8, including X25519MLKEM768 (0x11EC), x25519, secp256r1, x448, secp384r1, secp521r1, ffdhe2048, ffdhe3072
	Key shares sent	2 — X25519MLKEM768 (1216 bytes) and x25519 (32 bytes)
3	Group selected	x25519 (quantum vulnerable)
4	Signature algorithms offered	16, none post-quantum
5	Algorithm selected	rsa_pss_rsae_sha256 (quantum vulnerable)

The client offered a post-quantum hybrid group and pre-emptively sent both a hybrid key share and a classical fallback. The server selected the fallback. The 1216-byte hybrid share is the concatenation of an ML-KEM-768 encapsulation key (1184 bytes) with an X25519 public key (32 bytes); the 32-byte share beside it is the classical offer alone, and the difference between the two is the wire cost of the post-quantum component.

The CertificateRequest offered sixteen signature algorithms, ECDSA over the NIST curves, Ed25519, Ed448, RSA-PSS and RSA-PKCS1 variants, and no post-quantum option. Client authentication is therefore classical by construction, independently of what the client might support.

Notes.

Both halves of the TLS protection on this channel – key agreement and client authentication – are pre-quantum. The channel carries key material whose purpose is resistance to quantum adversaries.

This is fully conformant. ETSI GS QKD 014 [2] states that the interface is carried over TLS and imposes no requirement on cipher suites, groups or signature algorithms. Nothing was misconfigured and no requirement was missed; there is no requirement.

The practical exposure here is modest. Keys in this topology expire after 600 seconds, which limits the value of a harvest-now-decrypt-later attack against the delivery channel specifically. The observation is architectural rather than a live risk: the cryptographic posture of the hop where quantum-derived key material crosses into classical infrastructure is left entirely to the implementation and is therefore invisible to a procuring operator unless separately specified and tested.

No cryptographic configuration was applied to the test client. The hybrid group was offered by the OpenSSL build bundled with CPython in the python:3.12-slim image. It appeared without being asked for, which is itself an observation about the relative maturity of the two ecosystems, given the effort required elsewhere in this study to obtain a working ETSI 014 client at all.

Reusability. Steps 3 and 5 constitute a two-field posture check applicable to any TLS endpoint, QKD or otherwise, and take under a minute against an existing capture. Run against vendor KME equipment, this test case becomes comparative rather than descriptive. The same method applied to a post-quantum TLS deployment is documented in the companion study **ppq-tls13-oqs-poc**, where the hybrid group is selected rather than declined. One instrument, two answers.

7. Test Case TC-06 – Failure condition discrimination

Objective. TC-02 established that a delivered key cannot be retrieved a second time. Several distinct conditions may prevent retrieval, and it is not evident from the specification whether a requesting SAE can tell them apart. Determine, from the interface alone, whether a key already delivered, a key that expired before retrieval, an identifier never issued, and a malformed identifier produce distinguishable responses.

The test client constructs all four conditions deliberately. None represents an operational event.

Preconditions. Key stream expiry temporarily reduced from 600 s to 120 s for the duration of the test, and restored afterwards. All requests are issued to kme-2 under the sae-2 identity, so that only the identifier supplied varies.

Procedure. Four *Get key with key IDs* requests, implemented in `tc06_failure_conditions.py` (provided in the GitHub repository):

Case Identifier supplied		Condition under test
A	Key retrieved once already	Consumed
B	Key issued to the master SAE, never fetched, left past expiry	Expired
C	00000000-0000-0000-0000-000000000000	Syntactically valid, never assigned
D	not-a-uuid	Malformed

Case B is issued first and left pending while cases A, C and D execute, so that the expiry interval elapses within a single run.

Expected result. Cases A, B and C indistinguishable, on the reasoning that an interface which distinguishes them reveals which identifiers have been issued. Case D expected to differ, being a parameter validation failure rather than a retrieval failure.

Observed result. The expectation was not met. Three distinct responses were returned across the four conditions.

Case Condition		Status	Message
A	Consumed	404	Key not found
B	Expired	403	Key expired
C	Never issued	404	Key not found
D	Malformed	400	Invalid parameter value

Cases A and C are byte-identical but for the identifier echoed in the details object. Case B is explicitly distinguished, with both a different status code and a different message. Case D is rejected by parameter validation, naming the field and its permitted format:

```
json

{"message": "Invalid parameter value",

"details": [{"parameter_name": "key_ID",

               "parameter_value": "not-a-uuid",

               "allowed_values": "UUID"}]}
```

stored_key_count was 1953 both before and after the run, equal to max_key_count. The key store was saturated and replenished consumption immediately, so no conclusion can be drawn about whether expiry is measured from key generation or from assignment. This remains open.

23743	185.401320700	3.227.206.55	192.168.178.20	HTTP/JSON	A	512	211	HTTP/1.1	404 Not Found	JSON (application/json)
23792	185.794074900	3.227.206.55	192.168.178.20	HTTP/JSON	C	512	212	HTTP/1.1	404 Not Found	JSON (application/json)
23820	186.173698000	3.227.206.55	192.168.178.20	HTTP/JSON	D	558	213	HTTP/1.1	400 Bad Request	JSON (application/json)
24576	277.688381900	2001:a61:3098:8a01:3ea6:2f...	2001:a61:3098:8a0...	HTTP		790	229	HTTP/1.1	400 Bad Request	(text/html)
24824	334.256930900	3.227.206.55	192.168.178.20	HTTP/JSON	B	510	236	HTTP/1.1	403 Forbidden	JSON (application/json)

Figure 11 – Responses to four distinct retrieval failure conditions. A consumed key and an identifier never issued return identical responses; an expired key is reported explicitly with a different status code; a malformed identifier is rejected by parameter validation. Of these, only the 400 appears in the status codes defined by ETSI GS QKD 014 [2].


```

24824 334.256930900 3.227.206.55 192.168.178.20 HTTP/JSON B 510 236 HTTP/1.1 403 Forbidden JSON (application/json)
Name: 24824: Packet, 510 bytes on wire (4080 bits), 510 bytes captured (4080 bits) on interface \Device\NPF_{95BAF7AF-9C6F-474B-B8CD-C1E3DD54D48E}, id 0
Ethernet II, Src: AVMAudiovisu_87:39:f2 (3c:a6:2f:87:39:f2), Dst: CloudNetwork_0e:82:65 (fc:b0:de:0e:82:65)
Internet Protocol Version 4, Src: 3.227.206.55, Dst: 192.168.178.20
Transmission Control Protocol, Src Port: 443, Dst Port: 59958, Seq: 3839, Ack: 3029, Len: 456
Transport Layer Security
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
Object
  Member: message
    [Path with value: /message:Key expired]
    [Member with value: message:Key expired]
    String value: Key expired
    Key: message

```

Figure 12 – Key Expiry in case B. A key issued to the master SAE and left unfetched past the key stream expiry returns HTTP 403 with the message "Key expired", distinct in both status code and message from the 404 returned for a consumed or never-issued identifier.

```

23743 185.401320700 3.227.206.55 192.168.178.20 HTTP/JSON A 512 211 HTTP/1.1 404 Not Found JSON (application/json)
23792 185.794074900 3.227.206.55 192.168.178.20 HTTP/JSON C 512 212 HTTP/1.1 404 Not Found JSON (application/json)
Frame 23792: Packet, 512 bytes on wire (4096 bits), 512 bytes captured (4096 bits) on interface \Device\NPF_{95BAF7AF-9C6F-474B-B8CD-C1E3DD54D48E}, id 0
Ethernet II, Src: AVMAudiovisu_87:39:f2 (3c:a6:2f:87:39:f2), Dst: CloudNetwork_0e:82:65 (fc:b0:de:0e:82:65)
Internet Protocol Version 4, Src: 3.227.206.55, Dst: 192.168.178.20
Transmission Control Protocol, Src Port: 443, Dst Port: 50419, Seq: 3839, Ack: 3029, Len: 458
Transport Layer Security
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
Object
  Member: message
    [Path with value: /message:Key not found]
    [Member with value: message:Key not found]
    String value: Key not found
    Key: message

```

Figure 13 – Consumed and never-issued are indistinguishable (cases A and C). A key already delivered and a syntactically valid identifier that was never assigned return identical responses: HTTP 404 with the message "Key not found", differing only in the identifier echoed in the details object. The interface therefore reveals nothing about whether an identifier was ever issued.

Notes.

Consumed and never-issued are indistinguishable; expiry is reported explicitly. A client receiving 403 knows to request a fresh key; a client receiving 404 knows that retrying is pointless. Both responses are actionable. The fourth condition, a key whose assignment has not propagated between two separate key management systems, cannot arise on a hosted platform where both KMEs are the same system, and remains untested.

The specified error set. ETSI GS QKD 014 [2] defines the status codes 400, 401 and 503. Neither 403 nor 404 appears in that set. Both are nevertheless load-bearing in this implementation: 404 carries the key store post-condition that the standard mandates, and 403 carries expiry, which the standard also permits.

This is the substantive finding of the test case, and it is a property of the specification rather than of the implementation. **The standard requires that a delivered key be removed from the key store and provides no defined means of reporting that condition to the requesting SAE.** The same applies to expiry. An implementation must therefore invent its own signalling for behaviour the standard mandates, and two implementations may reasonably invent differently.

Departure as improvement. The specification defines 401 for an unauthorized request. In TC-04, cases B and C, a valid certificate belonging to an SAE not authorized for the requested key, returned 404 rather than 401. A conformant 401 would confirm that the identity exists but lacks authorisation; the 404 returned reveals nothing. The departure from the specified error model is, in this instance, an information-disclosure improvement over it.

Residual oracle. Distinguishing 403 from 404 confirms that a given identifier was once a validly assigned key. In practice this is of negligible value: a UUID carries approximately 122 bits of entropy, so enumeration is infeasible, and any party holding a genuine identifier obtained it legitimately. It is recorded as a design observation, not as a weakness.

8. Test Case TC-07 – Connection reuse and handshake cost

Objective. ETSI GS QKD 014 [2] specifies HTTPS as the transport and says nothing about connection management. Determine whether the KME supports HTTP connection reuse and measure the cost of the mutual TLS handshake relative to the request it carries. The question is a dimensioning one: whether an application requesting keys at a given rate incurs a full mutual TLS handshake per request.

The test client constructs all three conditions deliberately. *Get status* is used throughout, so no key material is consumed, and the test may be repeated without affecting the key store.

Preconditions. Standard topology, key expiry restored to 600 s. All requests are issued to kme-1 under the sae-1 identity.

Procedure. Three parts, implemented in tc07_connection_reuse.py (provided in the GitHub repository):

Part	Method	Determines
1	5 requests, a fresh connection each	Baseline; handshake time separated from request time
2	5 requests over a single connection	Whether reuse is possible at all
3	1 request with an explicit Connection: keep-alive header	Whether the server honours or overrides the client's preference

Part 1 measures the TCP connect and TLS handshake separately from the request/response exchange, by timing connect() and the request independently.

Expected result. Every response recorded earlier in this study carried Connection: close. Reuse was therefore expected to be unavailable, and part 2 expected to fail on the second request.

Observed result. The expectation was not met. Connection reuse is supported.

Part	Result
1	5 × HTTP 200, Connection: keep-alive on every response
2	5 of 5 requests served over a single connection
3	HTTP 200, Connection: keep-alive, no Keep-Alive parameters returned

Measurement	Value
Median handshake (TCP + mutual TLS)	240.9 ms
Median request/response	407.2 ms
Handshake share of a cold request	37 %

Filtering the capture on `tcp && tls.handshake.type == 1` returns **seven ClientHello messages against eleven requests issued**: five in part 1 where a fresh connection was opened each time, one in part 2 serving five requests, and one in part 3. Handshakes are not proportional to requests.

The first handshake of part 1 took 454.6 ms against approximately 240 ms for the remainder; the difference is DNS resolution and cold start, and the median is the representative figure.

Notes.

Correction to TC-01. The notes to TC-01 record Connection: close on every response and treat it as a property of the KME. It is not. The header was present in the **request** emitted by Python-urllib/3.12, which does not implement connection reuse; the server complied with the client's stated preference. The test client used here (`http.client`) sends no such header, and the server's default is keep-alive. The earlier observation was an artefact of the test client.

Where the cost lies. The handshake is the smallest of the two costs. At approximately 240 ms it corresponds to two round trips, consistent with TCP setup followed by a single-round-trip TLS 1.3 handshake, which places the round-trip time near 100 ms. A single *Get status* nevertheless takes 407 ms, so roughly 300 ms of each request is server-side processing rather than network transit. Across

five requests, reuse reduces total elapsed time from approximately 3.3 s to 2.3 s, a saving of around 30 %, not the order-of-magnitude effect that a handshake-per-request model would imply.

Limits of these figures. All measurements were taken from a container in Munich against a hosted platform over the public internet and include internet latency and the platform's own service time. They are not representative of vendor equipment on an operator network. The transferable observation is the ratio and the qualitative result, not the milliseconds.

What this means for the standard. Connection management is unspecified in ETSI GS QKD 014 [2]. This implementation supports reuse; another need not. The achievable key request rate therefore depends on behaviour the standard does not define, and which cannot be determined from the specification alone. It must be measured against each implementation. That is the finding, and it is narrower than the handshake-cost concern this test case was written to investigate.

Time	Source	Destination	Protocol	Length	tcp.str	Info
302 28.754304900	192.168.178.20	3.227.206.55	TLSv1.3	199 16	Client Hello	(SNI=kme-1.acct-4627.etsi-qkd-api.qukaydee.com)
325 29.420361400	192.168.178.20	3.227.206.55	TLSv1.3	199 17	Client Hello	(SNI=kme-1.acct-4627.etsi-qkd-api.qukaydee.com)
344 30.089709300	192.168.178.20	3.227.206.55	TLSv1.3	199 18	Client Hello	(SNI=kme-1.acct-4627.etsi-qkd-api.qukaydee.com)
367 30.755539700	192.168.178.20	3.227.206.55	TLSv1.3	199 19	Client Hello	(SNI=kme-1.acct-4627.etsi-qkd-api.qukaydee.com)
390 31.442336700	192.168.178.20	3.227.206.55	TLSv1.3	199 20	Client Hello	(SNI=kme-1.acct-4627.etsi-qkd-api.qukaydee.com)
409 32.102029900	192.168.178.20	3.227.206.55	TLSv1.3	199 21	Client Hello	(SNI=kme-1.acct-4627.etsi-qkd-api.qukaydee.com)
478 34.538290500	192.168.178.20	3.227.206.55	TLSv1.3	199 22	Client Hello	(SNI=kme-1.acct-4627.etsi-qkd-api.qukaydee.com)

Figure 14 – Connection reuse across eleven requests. Seven TCP conversations served eleven requests: five connections opened individually in part 1, one connection carrying five requests in part 2, and one in part 3. The mutual TLS handshake is amortised across a reused connection rather than repeated per request.

9. Test Case TC-08 – Key store exhaustion

Objective. ETSI GS QKD 014 [2] exposes the key store as a single counter and defines the status code 503 for a server-side error. Determine what a KME returns when the key store is depleted, whether it degrades gracefully or fails outright, and at what rate the store replenishes. The question is an operational one: what an application encounters when demand exceeds the rate at which the quantum layer produces key material.

The test client drains the store deliberately by requesting key material faster than the key stream produces it. The condition does not represent an operational event.

Preconditions. Key store saturated at `stored_key_count = max_key_count = 1953`, confirmed before the run. Key stream at 1000 bits/s, key expiry 600 s. Draining leaves the topology without key material for approximately sixteen minutes, so no other test case may run concurrently.

Procedure. Implemented in `tc08_exhaustion.py` (provided in the GitHub repository):

Phase	Method
Baseline	<i>Get status</i> to record <code>key_size</code> , <code>max_key_per_request</code> , <code>stored_key_count</code> , <code>max_key_count</code> (100 keys x 512 bits)
Drain	Successive <i>Get key</i> requests of <code>max_key_per_request</code> keys at <code>key_size</code> , each followed by <i>Get status</i> , until a request fails, to poll key store state.
Recovery	<i>Get status</i> sampled every 15 s for eight samples, to derive the replenishment rate

All requests are issued to `kme-1` under the `sae-1` identity.

Expected result. Depletion expected to produce HTTP 503, that being the status code the specification defines for a server-side condition. Partial delivery, fewer keys returned than requested, considered possible, the response format permitting it.

Observed result. The store was depleted after twenty requests. The expected status code was not returned.

Measurement	Value
Requests served in full	20
Keys delivered	2000
Partial deliveries	none
Status on depletion	400
Message	Not enough key material available

Every successful request returned exactly the 100 keys requested. `stored_key_count` fell from 1953 to 48 across the drain; the difference between 1905 and the 2000 keys delivered corresponds to material generated during the run. The twenty-first request failed:

```
json
{"message": "Not enough key material available",
 "details": [{"key_stream_ID": 385,
               "key_bits_requested": 51200,
               "key_bits_available": 25808}]}
```

Replenishment was linear across the sampling window:

Elapsed	<code>stored_key_count</code> (information received in get status response)
15.9 s	81
47.6 s	143

Elapsed	stored_key_count (information received in get status response)
79.6 s	205
128.3 s	300

The observed rate was 1.95 keys/s, equal to **997 bits/s** against the 1000 bits/s configured, matching to within sampling error. A request for 100 keys therefore cannot succeed for approximately 27 seconds after depletion, and replenishment from empty to max_key_count requires approximately 16 minutes.

The test case was executed twice. Both runs delivered 2000 keys in 20 full requests, failed identically on the twenty-first, and replenished at 1.95 keys/s. The two runs differed only in the residual key_bits_available reported at failure (25808 and 34764), reflecting how full the store was at the moment each run began.

Notes.

The status code is defined but semantically inverted. Unlike the codes observed in TC-06, 400 does appear in the specification's set. However, HTTP 400 signifies a malformed request that should not be repeated, whereas the request here was entirely valid and retrying after a delay is precisely the correct client behaviour: the store refills at a known and advertised rate. HTTP 503, which the specification also defines and which carries retry-later semantics, would have conveyed the opposite and correct instruction. For an application with automated retry logic, the distinction determines whether it backs off or abandons the request.

The error object is the most informative response in the study. The details object reports key_bits_requested and key_bits_available, allowing a client to compute exactly what it could have requested successfully. None of these fields appear in the error format defined by ETSI GS QKD 014 [2]; they are a vendor extension. This is the only point at which the interface exposes actionable key store state, and it does so only on failure.

All-or-nothing delivery. Material for approximately 50 keys was available when a request for 100 was refused. The response format defined by the standard permits returning fewer keys than requested, so partial delivery would have been conformant. This implementation elects hard failure over graceful degradation. The standard does not state which behaviour is correct, and two implementations may reasonably differ.

The key store is a bit pool. $\text{max_key_count } 1953 \times \text{key_size } 512 = 999\,936$ bits, against a store capacity reported by the platform as 1 000 000 bits. stored_key_count is therefore a derived, presentational figure, available bits divided by key_size, floored, and the error object's

key_bits_available is the underlying quantity. This is consistent with *Get key* accepting size=1024 in TC-01 despite key_size being advertised as 512: key material is cut to length from a continuous pool rather than issued as fixed-size objects. ETSI GS QKD 014 exposes only the derived counts, so an operator observes keys where the KME accounts bits.

Saturation is invisible. At baseline stored_key_count equalled max_key_count. The store was full and material produced by the key stream in excess of capacity is discarded. The interface provides no indication that this is occurring, and no means of determining how much generated material is being lost. Store capacity relative to link rate is therefore a dimensioning parameter that cannot be assessed through the API.

```
GET /api/v1/keys/sae-2/status HTTP/1.1
Accept-Encoding: identity
Host: kme-1.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Connection: close

HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 08 Aug 2026 11:58:21 GMT
Content-Type: application/json
Content-Length: 264
Connection: close
Vary: Accept, Cookie
Allow: GET, HEAD, OPTIONS
[...]
{"source_KME_ID":"kme-1","target_KME_ID":"kme-2","master_SAE_ID":"sae-1","slave_SAE_ID":"sae-2","key_size":512,"stored_key_count":1665,"max_key_count":1953,"max_key_per_request":100,"max_key_size":100000,"min_key_size":1,"max_SAE_ID_count":9,"status_extension":{}}
```

Figure 15a — Key store state as reported by *Get status*. The response carries the capability set against which requests are validated, key_size, max_key_per_request, min_key_size, max_key_size, and stored_key_count, the only key store observable available in normal operation.

Time	Source	Destination	Protocol	Length	tcp.strez Info
1423	63.594030300	3.227.206.55	192.168.178.20	HTTP/JSON	561 61 HTTP/1.1 400 Bad Request

Name 1423: Packet, 561 bytes on wire (4488 bits), 561 bytes captured (4488 bits) on interface \Device\NPF_{958AF7AF-9C6F-474B-B8CD}

Ethernet II, Src: AVMAudiovisu_87:39:f2 (3c:a6:2f:87:39:f2), Dst: CloudNetwork_0e:82:65 (fc:b0:de:0e:82:65)

Internet Protocol Version 4, Src: 3.227.206.55, Dst: 192.168.178.20

Transmission Control Protocol, Src Port: 443, Dst Port: 57139, Seq: 3838, Ack: 3005, Len: 507

Transport Layer Security

Hypertext Transfer Protocol

JavaScript Object Notation: application/json

Object

- Member: message
 - [Path with value: /message:Not enough key material available]
 - [Member with value: message:Not enough key material available]
 - String value: Not enough key material available
 - Key: message
 - [Path: /message]
- Member: details
 - Array
 - Object
 - Member: key_stream_ID
 - [Path with value: /details/[]/key_stream_ID:385]
 - [Member with value: key_stream_ID:385]
 - Number value: 385
 - Key: key_stream_ID
 - [Path: /details/[]/key_stream_ID]
 - Member: key_bits_requested
 - [Path with value: /details/[]/key_bits_requested:51200]
 - [Member with value: key_bits_requested:51200]
 - Number value: 51200
 - Key: key_bits_requested
 - [Path: /details/[]/key_bits_requested]
 - Member: key_bits_available
 - [Path with value: /details/[]/key_bits_available:25808]
 - [Member with value: key_bits_available:25808]
 - Number value: 25808
 - Key: key_bits_available
 - [Path: /details/[]/key_bits_available]

Figure 15b – Key store state at depletion. The twenty-first request for 100 keys is refused with HTTP 400 and the message "Not enough key material available". The details object reports the bits requested and the bits available, the only point in the interface at which key store state is exposed in its underlying unit.

10. Test Case TC-09 – POST request form and extension handling

Objective. ETSI GS QKD 014 [2] defines a POST form of *Get key* and *Get key with key IDs*, carrying a JSON request body. The body admits parameters unavailable in the GET form, among them the extension mechanism: *extension_mandatory*, which a KME **shall** reject if it does not support the parameter, and *extension_optional*, which it may ignore. Only the GET form has been exercised elsewhere in this study. Determine whether the POST form is available, whether several key identifiers may be retrieved in one request, and whether the extension mechanism behaves as specified.

The extension parameters supplied are fabricated and cannot be supported by any implementation. All conditions are constructed by the test client.

Preconditions. Standard topology, key store replenished after TC-08. All requests carry Content-Type: application/json.

Procedure. Implemented in `tc09_post_extensions.py` (provided in the GitHub repository):

Case Request		Condition under test
A	POST <i>Get key</i> , body {"number": 2, "size": 512}	Availability of the POST form
B	POST <i>Get key with key IDs</i> , two identifiers in one body	Batch retrieval
C	POST <i>Get key</i> with an unknown <i>extension_optional</i>	Unknown optional extension ignored
D	POST <i>Get key</i> with an unknown <i>extension_mandatory</i>	Unknown mandatory extension rejected

Cases A, C and D are issued to kme-1 under the sae-1 identity; case B to kme-2 under sae-2.

Expected result. The POST form available and equivalent to the GET form. Case C accepted, the unknown optional extension ignored. Case D rejected, per the requirement in the specification.

Observed result. All four cases behaved as specified.

Case Status Result

A	200	Two keys returned
---	-----	-------------------

B	200	Both keys returned in one response
---	-----	------------------------------------

C	200	Unknown optional extension ignored, key delivered
---	-----	---

D	400	Unknown mandatory extension
---	-----	-----------------------------

Case B returned both keys, byte-identical to those delivered in case A, in a single response:

```
json
{"key_IDs": [{"key_ID": "814e7763-...", {"key_ID": "a5646a06-..."}}]}
```

Case D was refused, with the offending parameter echoed back in the error object:

```
{"message": "Unknown mandatory extension",
"details": [{"extension_item": {"eclaireur_test_mandatory": "must-reject"}}]}
```

Notes.

The first mandatory requirement confirmed satisfied. Every earlier test case in this study either verified behaviour the specification describes without mandating, or found behaviour the specification does not define at all. Case D tests an explicit requirement, that a KME reject a mandatory extension it does not support, and the implementation satisfies it. The extension mechanism therefore operates as a negotiation rather than as a silent hazard: a vendor-specific feature marked mandatory cannot be quietly disregarded by a peer that lacks it. Given that vendor extensions are the principal route by which conformant implementations diverge, this is a load-bearing requirement and it holds here.

The same status code, used correctly and incorrectly. HTTP 400 appears both here and in TC-08. In this test case its use is correct: the request was genuinely unacceptable and must not be repeated unchanged. In TC-08 the same code was returned for a valid request that should be retried once the

key store replenishes. The specification's error vocabulary is small enough that a single code carries both meanings, and nothing in the document distinguishes them.

Batch retrieval and connection cost. Case B retrieved two keys in one request. Combined with the connection reuse established in TC-07, a client fetching a batch of keys need incur neither one connection nor one request per key. The specification places no obligation on a KME to support either, so the achievable key acquisition rate remains implementation-dependent, but where both are available, as here, the transaction cost is substantially lower than the GET form alone would suggest.

Untested extension behaviour. Only unknown extensions were exercised. Whether the platform supports any extensions of its own, and how a supported mandatory extension is negotiated, is not determined. No extension registry exists for ETSI GS QKD 014, so an implementation has no means of discovering which extensions a peer supports other than by attempting them and observing the rejection.

```
POST /api/v1/keys/sae-2/enc_keys HTTP/1.1
Accept-Encoding: Identity
Content-Length: 96
Host: kme-1.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Content-Type: application/json
Connection: close
{"number": 1, "size": 512, "extension_mandatory": [{"eclaireur_test_mandatory": "must-reject"}]}

HTTP/1.1 400 Bad Request
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 08 Aug 2026 13:46:54 GMT
Content-Type: application/json
Content-Length: 115
Connection: close
Vary: Accept, Cookie
Allow: GET, POST, HEAD, OPTIONS
[...]
{"message": "Unknown mandatory
extension", "details": [{"extension_item": {"eclaireur_test_mandatory": "must-reject"}]}
```

Figure 16 – Rejection of an unknown mandatory extension. A POST *Get key* request carrying a fabricated `extension_mandatory` parameter is refused with HTTP 400, the error object echoing the unsupported item. ETSI GS QKD 014 [2] requires this behaviour; unknown `extension_optional` parameters are ignored and the request proceeds.

11. Test Case TC-10 – Multicast key delivery

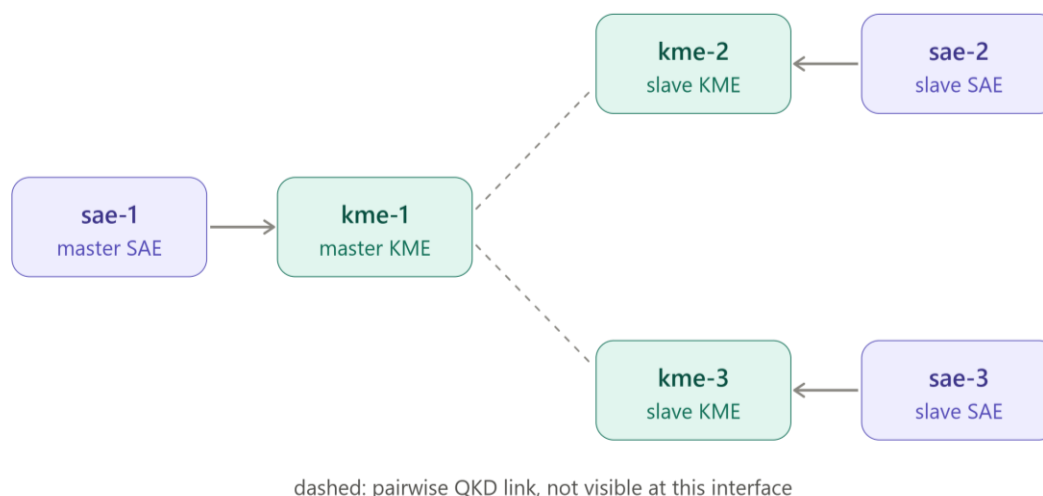


Figure 1b – Extended topology for TC-10. A third KME and SAE are added, and the key stream spans all three SAEs. The master SAE reaches each slave over a separate pairwise link. Solid arrows are ETSI 014 requests over mutual TLS; dashed links carry the key material shared between KMEs and are not exposed by the interface.

The two links “kme-1 - kme-2” and “kme-1 - kme-3” are independent. Key material identical at both slaves therefore cannot have been generated jointly.

Objective. ETSI GS QKD 014 [2] permits a master SAE to designate additional slave SAEs for a key, through the `additional_slave_SAE_IDs` parameter of the POST form of *Get key*. The capability is advertised in *Get status* as `max_SAE_ID_count`, reported as 9 by this platform. Determine whether the parameter is honoured, that is, whether several slave SAEs, each served by a different KME, retrieve identical key material, and what consequences follow for the key store post-condition established in TC-02.

Acceptance of the parameter is not evidence of delivery. This test case verifies delivery.

Preconditions. Extended topology (Figure 1b): a third KME kme-3 and a third SAE sae-3 were added, and a second key stream (ID 387) configured spanning sae-1, sae-2 and sae-3, alongside the existing pairwise stream (ID 385). The certificate for sae-3 was generated from the client root CA already registered with the platform. Both key streams replenished to `max_key_count` before the run.

Procedure. Implemented in `tc10_multicast.py` (provided in the GitHub repository):

Step Action	Endpoint	Identity
-------------	----------	----------

Step	Action	Endpoint	Identity
1	POST <i>Get key</i> , body {"number": 1, "size": 512, "additional_slave_SAE_IDs": ["sae-3"]}	kme-1 /keys/sae-2/enc_keys	sae-1
2	<i>Get key with key IDs</i> (with Key ID returned in step 1)	kme-2 /keys/sae-1/dec_keys	sae-2
3	<i>Get key with key IDs</i> (with Key ID returned in step 1)	kme-3 /keys/sae-1/dec_keys	sae-3
4	Byte-compare all three key values	—	—

Only the key identifier is conveyed between the three SAE identities; it is a variable within the test client.

Expected result. Steps 2 and 3 both succeed, returning key material byte-identical to that delivered in step 1.

Observed result. Pass.

Step	Actor	Status	Key matches master
1	sae-1 at kme-1	200	—
2	sae-2 at kme-2	200	yes
3	sae-3 at kme-3	200	yes

Key b2a7f18f-5a72-43cb-a008-f5d82b71b77b was delivered identically to three parties across three KMEs. The base64 key value returned at each of the three endpoints was identical.

Notably, **step 3 succeeded after step 2 had already retrieved the key**. The key was not removed from the store on first delivery. This is explainable by the fact that kme-1 received the “additional_slave_SAE_IDs” in the get key message from sae-1.

Notes.

The key store post-condition does not apply per retrieval. TC-02 established that a delivered key is removed from the key store and cannot be retrieved a second time. In the multicast case that behaviour does not hold: the key remained available to the second designated slave after the first had collected it. This is necessary for the feature to function at all, but ETSI GS QKD 014 [2] states the post-condition for the single-slave case and does not specify how it applies when several slaves are entitled to the same key. When the key is released, after the last designated slave has collected it, after a timeout, or only on expiry, is therefore an implementation decision. Two implementations may reasonably differ, and an application cannot determine which behaviour it faces from the interface.

A multicast key is not a pairwise secret. Three parties hold identical key material. The key cannot be used to secure a confidential channel between sae-1 and sae-2, because sae-3 holds the same bytes. This is inherent to the feature and appropriate for its intended use: group communication, multicast payloads, multi-site decryption of a single stream.

The interface, however, gives the slave no indication of it. The key container returned to sae-2 is structurally identical whether the key was designated for one slave or for nine; nothing in the response reports how many parties hold it. An application that requests a key with additional slaves and subsequently uses it for a point-to-point channel would suffer a confidentiality failure that the interface provides no means of detecting. Only the master SAE knows how many parties were designated.

Multicast implies key relay in a physical deployment. Quantum key distribution produces correlated material pairwise. In the topology of Figure 1b, kme-1 shares one pool with kme-2 and a separate, independent pool with kme-3; the probability of identical material arising in both is negligible. For a single key to reach both slaves, the master KME must select a key from one pool and transport it to the second peer, protected by a key drawn from the second pool, the trusted-node relay construction.

An application developer setting additional_slave_SAE_IDs therefore invokes key relay, with the trust assumptions that entails, through a parameter of a key delivery API. ETSI GS QKD 014 advertises the capability through max_SAE_ID_count and specifies neither how it is to be realised nor what trust assumptions attach to it.

This inference is drawn from the properties of pairwise quantum key distribution and from the topology, not from observation. The platform under test has no physical quantum links and may write identical material into all three key stores directly, which no physical system can do. The relay is what a deployment would be required to perform; it was not observed here.


```

POST /api/v1/keys/sae-2/enc_keys HTTP/1.1
Accept-Encoding: identity
Content-Length: 65
Host: kme-1.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Content-Type: application/json
Connection: close

{"number": 1, "size": 512, "additional_slave_SAE_IDs": ["sae-3"]}
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 08 Aug 2026 13:50:32 GMT
Content-Type: application/json
Content-Length: 157
Connection: close
Vary: Accept, Cookie
Allow: GET, POST, HEAD, OPTIONS
[...]
{"keys":[{"key_ID":"b2a7f18f-5a72-43cb-a008-f5d82b71b77b","key":"I+uuu+DaE9cl5529RzQrFyA2jmf00HPBXmsOF
E9mWxCduStLJV/iWTA91AFot23ZHJnjgZxsbB62oJbUa+f03A=="}]}

GET /api/v1/keys/sae-1/dec_keys?key_ID=b2a7f18f-5a72-43cb-a008-f5d82b71b77b HTTP/1.1
Accept-Encoding: identity
Host: kme-2.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Connection: close

HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 08 Aug 2026 13:50:32 GMT
Content-Type: application/json
Content-Length: 157
Connection: close
Vary: Accept, Cookie
Allow: GET, POST, HEAD, OPTIONS
[...]
{"keys":[{"key_ID":"b2a7f18f-5a72-43cb-a008-f5d82b71b77b","key":"I+uuu+DaE9cl5529RzQrFyA2jmf00HPBXmsOF
E9mWxCduStLJV/iWTA91AFot23ZHJnjgZxsbB62oJbUa+f03A=="}]}

GET /api/v1/keys/sae-1/dec_keys?key_ID=b2a7f18f-5a72-43cb-a008-f5d82b71b77b HTTP/1.1
Accept-Encoding: identity
Host: kme-3.acct-4627.etsi-qkd-api.qukaydee.com
User-Agent: Python-urllib/3.12
Accept: application/json
Connection: close

HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 08 Aug 2026 13:50:33 GMT
Content-Type: application/json
Content-Length: 157
Connection: close
Vary: Accept, Cookie
Allow: GET, POST, HEAD, OPTIONS
[...]
{"keys":[{"key_ID":"b2a7f18f-5a72-43cb-a008-f5d82b71b77b","key":"I+uuu+DaE9cl5529RzQrFyA2jmf00HP
BXmsOFE9mWxCduStLJV/iWTA91AFot23ZHJnjgZxsbB62oJbUa+f03A=="}]}

```

Figure 17 - Multicast key delivery. The same key identifier and byte-identical key material returned at three endpoints: kme-1 under sae-1, kme-2 under sae-2, and kme-3 under sae-3. Only the identifier was conveyed between the SAE identities.

12. Outlook

This section is reflection rather than testing. Nothing in it is supported by the captures. It sets out what the preceding test cases suggest would need to be specified before a QKD network could be procured and operated by a network operator rather than assembled by a single vendor.

What the interface does not expose

ETSI GS QKD 014 [2] defines what a key store must *do* at the application boundary. It says nothing about what a key store must *be*.

The test cases make this visible from outside. TC-01 depends on both KMEs already holding identical key material before any request is issued, a precondition established entirely outside the interface. TC-02 shows a key leaving the store on delivery, but the store itself is observable only as the single `stored_key_count` scalar in *Get status*. TC-04 shows that where the identity trust boundary physically sits is an implementation choice the standard does not constrain.

TC-06 shows that the standard mandates the key store post-condition and provides no defined means of reporting it: the status codes carrying both consumption and expiry lie outside the set the specification defines. TC-08 shows the store is accounted in bits while the interface exposes only derived key counts, and that a saturated store discarding generated material is invisible at the interface. TC-10 shows the post-condition itself is conditional – a key designated for several slaves is not released on first delivery – a case the standard does not address.

Behind the three methods sits a **stateful, real-time key store**: key material, identifiers, master and slave SAE bindings, per-key state, expiry timestamps, peer KME identity. Two such stores, at opposite ends of a link, with no shared storage and only a classical channel between them, must agree. That is a **distributed consistency problem**, and it is where the engineering difficulty of a KME lies.

What a multi-vendor deployment would require

Interoperability between vendors divides into two cases of very different difficulty.

- a. **Two vendors' equipment at opposite ends of the same QKD link** requires agreement on the optical layer, the QKD protocol, and the full post-processing chain: sifting, error correction, privacy amplification.
- b. **Two vendors' key management systems meeting at a node** (“KMS-Border-Gateway”), each serving its own links, is the tractable case and the realistic procurement scenario: an operator building out regionally becomes multi-vendor by geography and tender cycle long before anyone asks two vendors to share a fibre pair. This is the boundary ETSI GS QKD 020 [5] addresses.

For that boundary to be operationally safe, several behaviours would need to be specified rather than left to implementations:

- **Key assignment semantics.** When one KME assigns a key to an SAE pair, does it wait for the peer to acknowledge before responding to the requesting SAE, or respond optimistically? Blocking costs latency on every request; optimistic response creates a window in which the slave SAE can present an identifier the peer has not yet recorded.
- **Reconciliation after a signalling interruption.** The quantum and classical channels can fail independently. Loss of the classical channel leaves both key stores producing material while their state diverges. Recovery requires a defined reconciliation rule (log), a designated initiator, and bounded work per round (for example a sequenced journal replayed from the peer's last acknowledged position for short outages, digest comparison over identifiers and states for longer ones, and a full epoch flush beyond a defined threshold).
- **Timers and retry bounds.** Reconnect backoff, acknowledgement timeouts, the threshold at which incremental replay gives way to full resynchronisation, and hysteresis on that threshold.
- **Client retry guidance.** TC-06 established that a consumed key and an unknown identifier are indistinguishable, while an expired key is reported explicitly. In a deployment with separate key management systems, a key whose assignment has not yet propagated would be indistinguishable from a consumed one, and the two require opposite client behaviour. Without an acknowledgement mechanism, a client cannot determine which it faces. TC-08 shows the same ambiguity in the status code itself: key store depletion, a condition that should be retried, is reported with a code whose semantics instruct the client not to.

None of these appear in ETSI GS QKD 014 [2]. Two conformant implementations may therefore differ in ways that surface only under load or after a link fault.

The deliverables addressing the operational layer

014 is deliberately narrow, and the operational layer is distributed across other deliverables:

Deliverable	Scope
GS QKD 004 [6]	Session-oriented streaming key delivery API
GS QKD 014 [2]	REST key delivery API (this report)
GS QKD 015 [3]	Control of QKD networks in SDN environments
GS QKD 018 [4]	Orchestration in SDN-enabled networks
GS QKD 020 [5]	Interoperable Key Management System API

ETSI GS QKD 020 V1.1.1 was published in June 2026 [5], and a second edition of 014 is in the ETSI work programme. The specification covering the tractable interoperability case therefore now exists.

It has not been assessed here; the questions in the previous paragraph on multivendor deployments are those to be checked.

What the standard specifies well

Two mechanisms were found to work exactly as specified. TC-03 confirms that the capability set advertised in *Get status* is binding, with errors that name the parameter and state its permitted range. TC-09 confirms the extension mechanism: an unknown extension_mandatory parameter is refused while an unknown extension_optional is ignored, so a vendor-specific feature marked mandatory cannot be silently disregarded by a peer that lacks it. Since vendor extensions are the principal route by which conformant implementations diverge, this is a load-bearing requirement, and it holds. No extension registry exists, however, so an implementation can discover which extensions a peer supports only by attempting them.

The gap that is not in any single document

The suite is a set of documents, not a release. Each deliverable specifies a slice, at independent version numbers, on independent timelines. There is no profile stating that a carrier-grade QKD network implements these documents, at these versions, with this conformance suite attached.

The contrast with mobile networking is instructive. A 3GPP release is a coherent, frozen, testable bundle, supported by interoperability testing campaigns and plugfests, against which an operator can procure and a vendor can certify. The QKD suite has no equivalent at the interface layer. GS QKD 016 provides a Common Criteria protection profile, but for the optical modules rather than the network interfaces.

This is not a criticism of any individual deliverable. It is an observation about what is required for a specification to become procurable, and it has a practical consequence visible in this exercise: no reference implementation, conformance suite or distributable image accompanies the standard. The environment used here exists because a third party built and hosts one. Absent that, every operator evaluating this technology repeats the work privately, per vendor. A published specification is a necessary condition for interoperability; it has never been a sufficient one.

The test cases in this report are a private, informal instance of what a conformance suite would provide publicly: defined procedures with defined expected results, applied identically to every implementation. Their value was demonstrated in the writing of this report. Three of the ten test cases returned results contradicting what had been inferred from reading the specification: expiry proved distinguishable from consumption, connection reuse proved available, and key store depletion returned a status code other than the one the specification's error set would suggest. A specification read carefully is not a specification tested.

13. Further Investigations and Test Cases

Requiring a different environment:

- The KME-to-KME channel, assessed against ETSI GS QKD 020 V1.1.1 [5].
- Trusted-node relay and multi-hop key delivery.

- The same test cases run against vendor equipment, where TC-04 and TC-05 become comparative instruments rather than single-endpoint characterisations.

14. Acknowledgement

This study was possible because **QuKayDee** provides a hosted, freely accessible ETSI GS QKD 014 [2] endpoint with a multi-KME topology, complete certificate handling, and clear documentation. It states plainly that it is **intended for education and API testing** and that its keys are not cryptographically secure, and it was used strictly within that scope. Every observation in this report concerning API structure, error behaviour and the mutual TLS identity model was obtainable only because the platform exists; a first-hand look at this interface would otherwise have required vendor equipment. **The findings concern the specifications and the ecosystem around it, not the platform.**

References & Normative Frameworks

QuKayDee Platform

[1] **QuKayDee Platform:** <https://qukaydee.com/pages/about>. QuKayDee is a Quantum Key Distribution (QKD) network simulator that runs in the cloud, with a focus on Application Programming Interfaces (APIs). Account can be freely activated. A tutorial section showing how to setup the infrastructure and a “getting started” section are available.

ETSI Specifications

[2] **ETSI GS QKD 014 v1.1.1:** Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API. A second edition of ETSI GS QKD 014 is in the ETSI work programme.
https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01.01.01_60/gs_qkd014v010101p.pdf

[3] **ETSI GS QKD 015 (SDN):** Quantum Key Distribution (QKD); Control Interface for Software Defined Networks.
https://www.etsi.org/deliver/etsi_gs/QKD/001_099/015/02.01.01_60/gs_QKD015v020101p.pdf

[4] **ETSI GS QKD 018:** Quantum Key Distribution (QKD); Orchestration Interface for Software Defined Networks.
https://www.etsi.org/deliver/etsi_gs/QKD/001_099/018/01.01.01_60/gs_qkd018v010101p.pdf

[5] **ETSI GS QKD 020, KMS interoperability, published June 2026:** Quantum Key Distribution (QKD); Protocol and data format of REST-based Interoperable Key Management System API.
https://www.etsi.org/deliver/etsi_gs/QKD/001_099/020/01.01.01_60/gs_QKD020v010101p.pdf

[6] **ETSI GS QKD 004:** Quantum Key Distribution (QKD); Application Interface.
https://www.etsi.org/deliver/etsi_gs/QKD/001_099/004/02.01.01_60/gs_qkd004v020101p.pdf

Tooling

[7] **QuKayDee client certificate generation scripts: Rijsman, B.** [qukaydee-generate-client-certificates](https://github.com/brunorijman/qukaydee-generate-client-certificates). Shell scripts generating a client root CA and SAE client certificates in the distinguished-name format expected by the QuKayDee platform. Used here to produce the sae-1, sae-2 and sae-9 credentials, and, with a second, unregistered CA, the rogue-sae-1 credential of TC-04.
<https://github.com/brunorijman/qukaydee-generate-client-certificates>